

Algorithm Analysis And Data Structures

Prof. Dr. Ayla ŞAYLI

<http://avesis.yildiz.edu.tr/sayli/>
sayli@yildiz.edu.tr

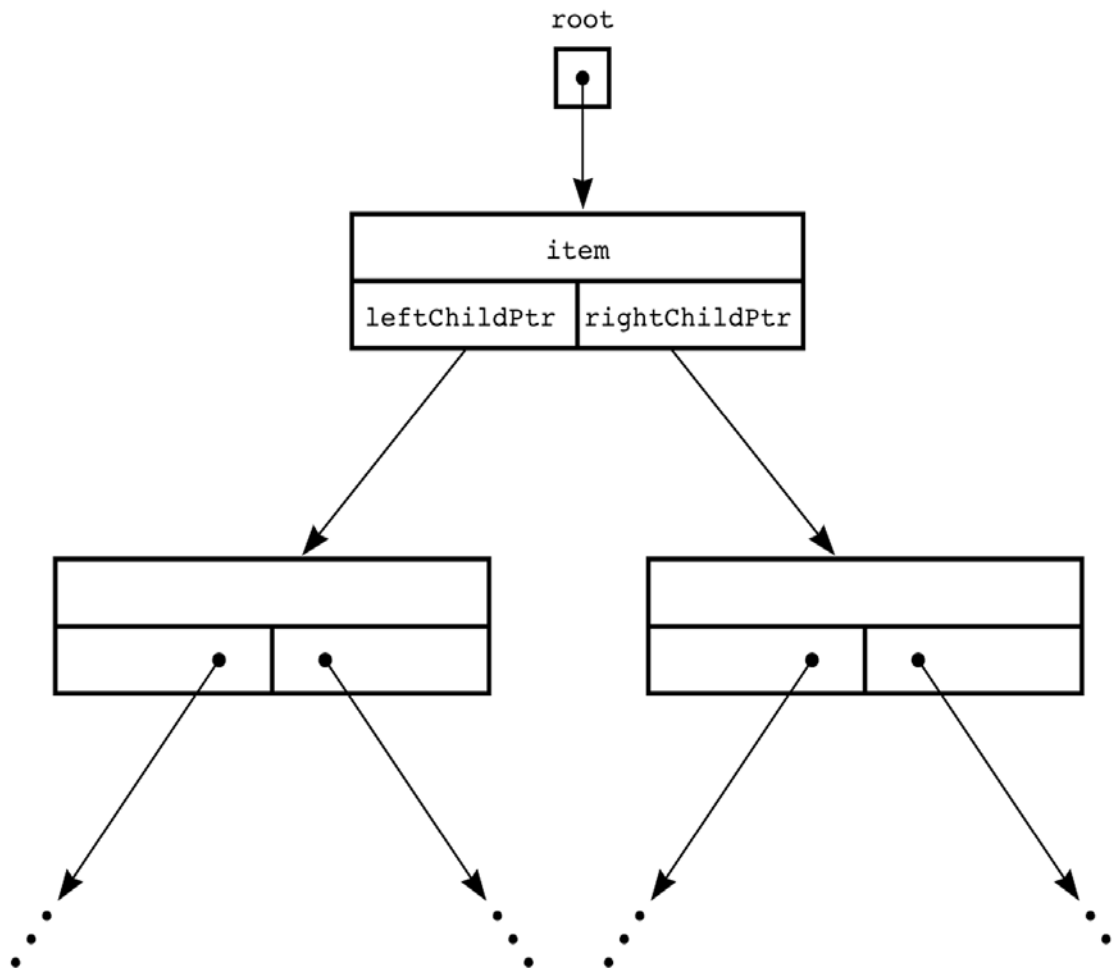
Implementation of Binary Trees

```
typedef struct tree_node *tree_ptr;
```

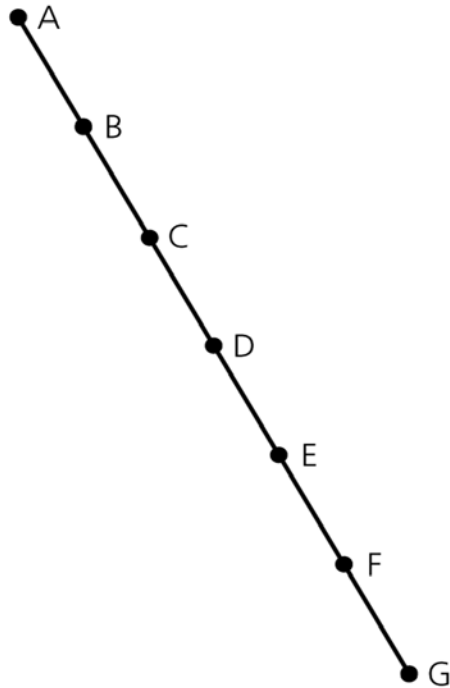
```
struct tree_node  
{  
    element_type element;  
    tree_ptr left;  
    tree_ptr right;  
};
```

```
typedef tree_ptr TREE;
```

Implementation of Binary Trees



Maximum and Minimum Heights of a Binary Tree



A maximum-height binary tree with seven nodes



(a)



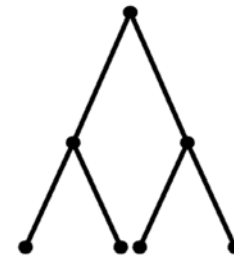
(b)



(c)



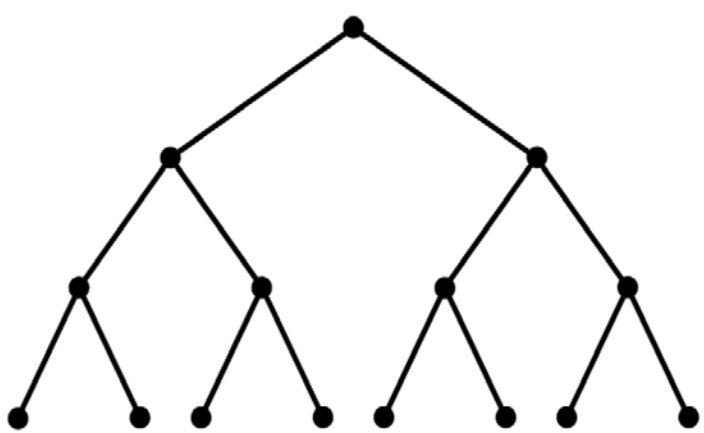
(d)



(e)

Some binary trees of height 2

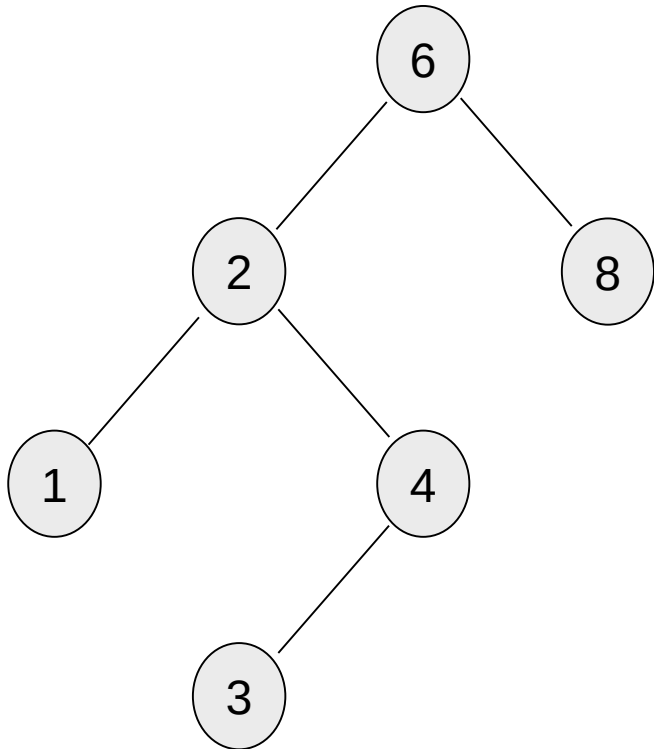
Counting the nodes in a full binary tree

	Level	Number of nodes at this level	Number of nodes at this and previous levels
	1	$1 = 2^0$	$1 = 2^1 - 1$
	2	$2 = 2^1$	$3 = 2^2 - 1$
	3	$4 = 2^2$	$7 = 2^3 - 1$
	4	$8 = 2^3$	$15 = 2^4 - 1$
• • • •	•	•	•
• • • •	•	•	•
• • • •	•	•	•
	h	2^{h-1}	$2^h - 1$

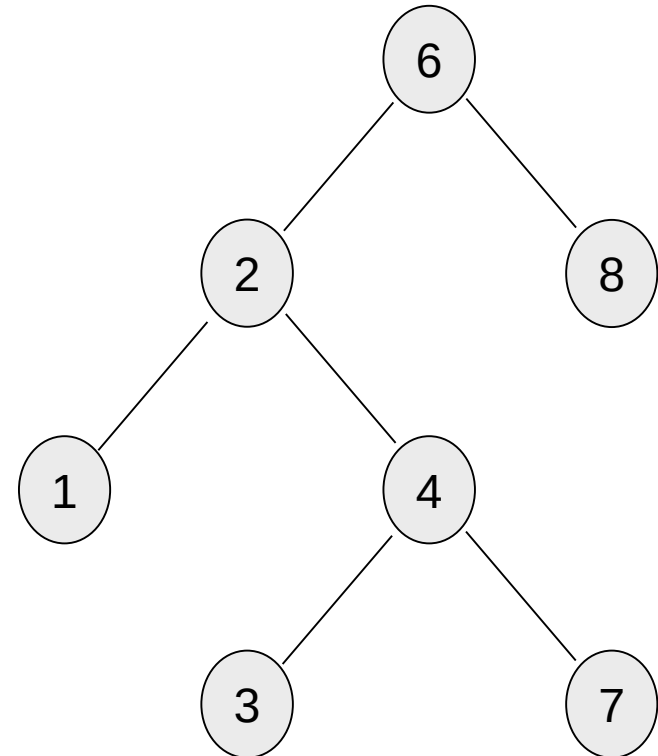
Binary Search Trees (BST)

- An important application of binary trees is their use in searching.
- *Binary search tree* is a binary tree in which every node X contains a data value that satisfies the following:
 - ❖ all data values in its left sub-tree are smaller than the data value in X
 - ❖ the data value in X is smaller than all the values in its right sub-tree.
 - ❖ the left and right sub-trees are also binary search trees.

Example

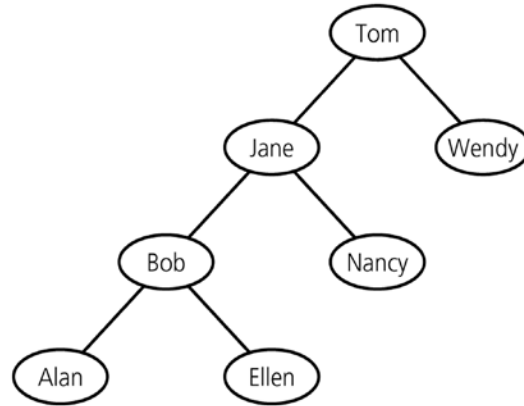


A binary search tree

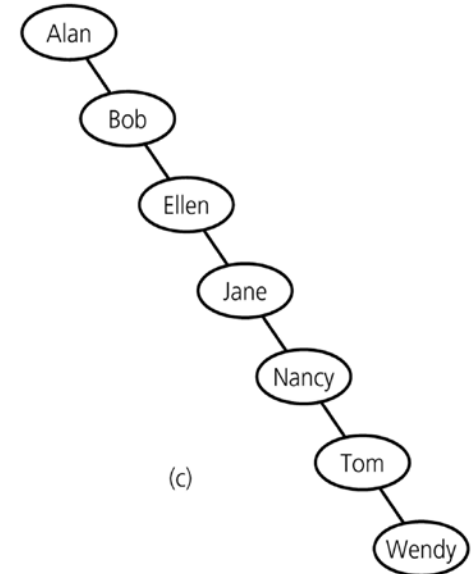


*Not a binary search tree,
but a binary tree*

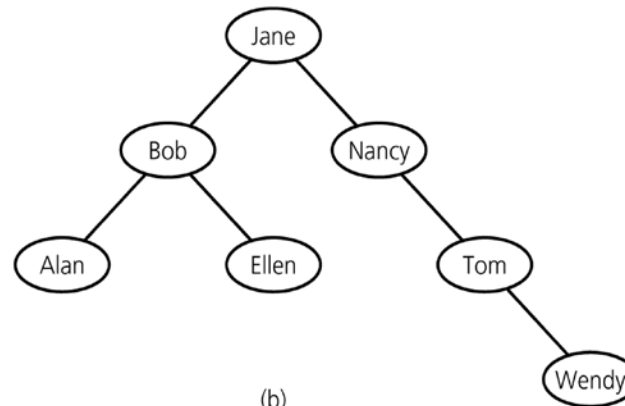
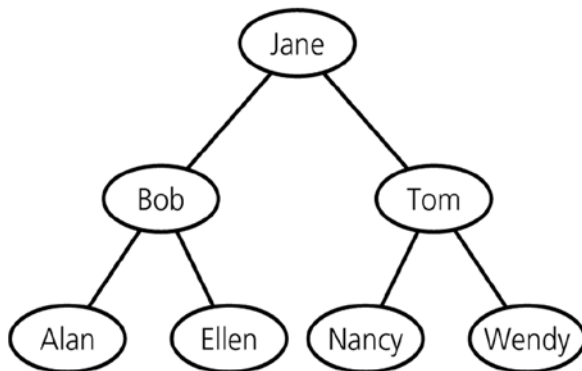
Binary Search Trees - containing same data



(a)



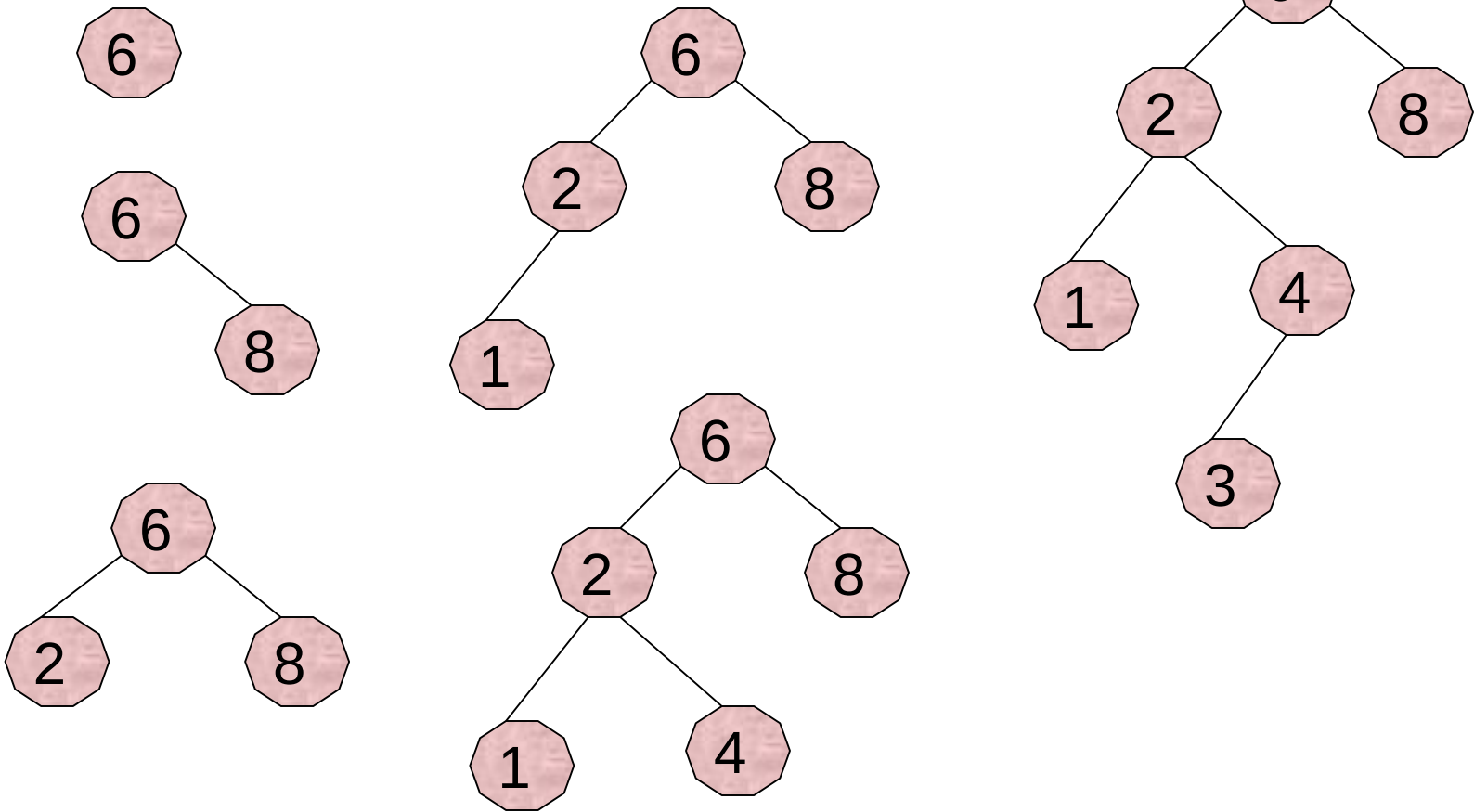
(c)



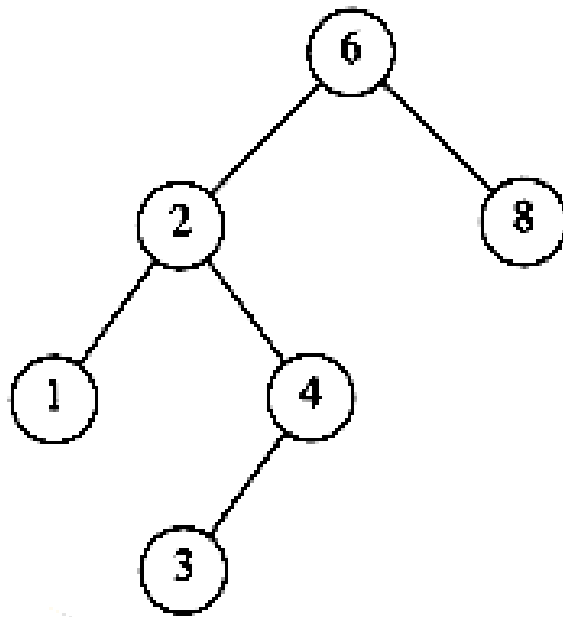
(b)

Insertion

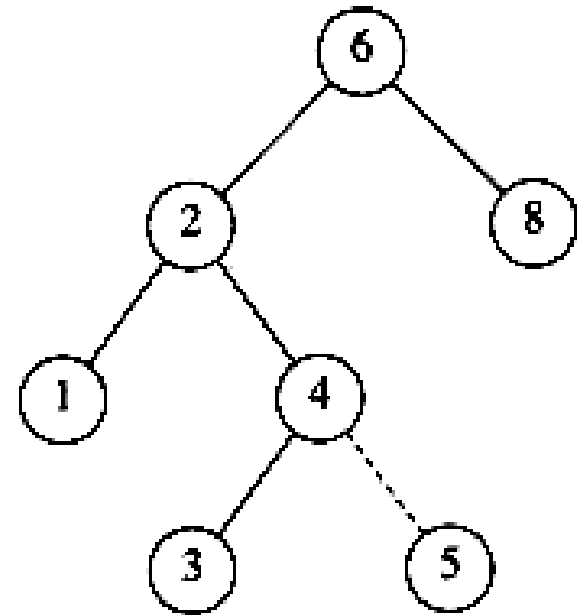
Example: 6, 8, 2, 1, 4, 3



Insertion



Insert 5



Structure of BST

```
typedef struct tree_node *tree_ptr;
```

```
struct tree_node  
{  
    element_type element;  
    tree_ptr left;  
    tree_ptr right;  
};
```

```
typedef tree_ptr SEARCH_TREE;
```

```

tree_ptr insert( element_type x, SEARCH_TREE T )
{
    if( T == NULL )
    {
        T = (SEARCH_TREE) malloc ( sizeof (struct tree_node) );
        if( T == NULL )
            fatal_error("Out of space!!!");
        else
        {
            T->element = x;
            T->left = T->right = NULL;
        }
    }
    elseif( x < T->element )
        T->left = insert( x, T->left );
    elseif( x > T->element )
        T->right = insert( x, T->right );
    return T;
}

```

Figure 4.22 Insertion into a binary search tree

```
SEARCH_TREE make_null ( void )  
{  
    return NULL;  
}
```

```
tree_ptr find( element_type x, SEARCH_TREE T )  
{  
    if( T == NULL )  
        return NULL;  
    if( x < T->element )  
        return( find( x, T->left ) );  
    else if( x > T->element )  
        return( find( x, T->right ) );  
    else  
        return T;  
}
```

```
tree_ptr find_min( SEARCH_TREE T )
{
    if( T == NULL )
        return NULL;
    else if( T->left == NULL )
        return( T );
    else
        return( find_min ( T->left ) );
}
```

```
tree_ptr find_max( SEARCH_TREE T )
{
    if( T != NULL )
        while( T->right != NULL )
            T = T->right;
    return T;
}
```

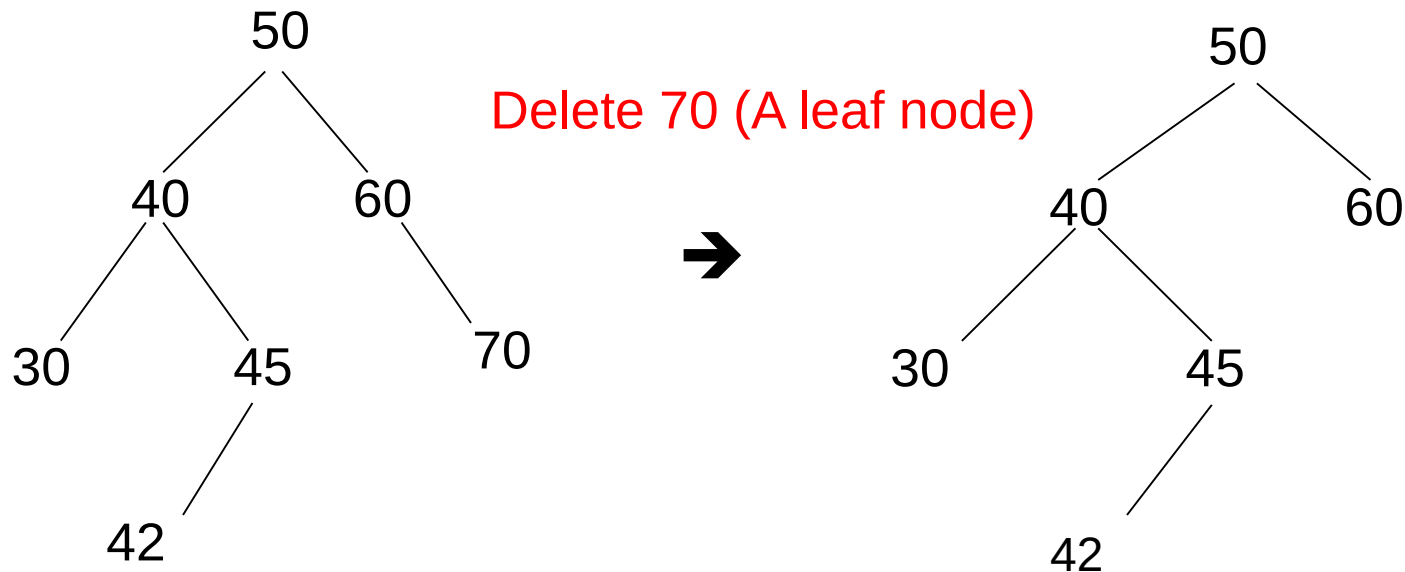
Deletion

There are three cases to consider:

- Deleting a leaf node
 - ❖ Replace the link to the deleted node by NULL.
- Deleting a node with one child:
 - ❖ The node can be deleted after its parent adjusts a link to bypass the node.
- Deleting a node with two children:
 - ❖ The deleted value must be replaced by an existing value that is either one of the following:
 - The largest value in the deleted node's left sub-tree
 - The smallest value in the deleted node's right sub-tree.

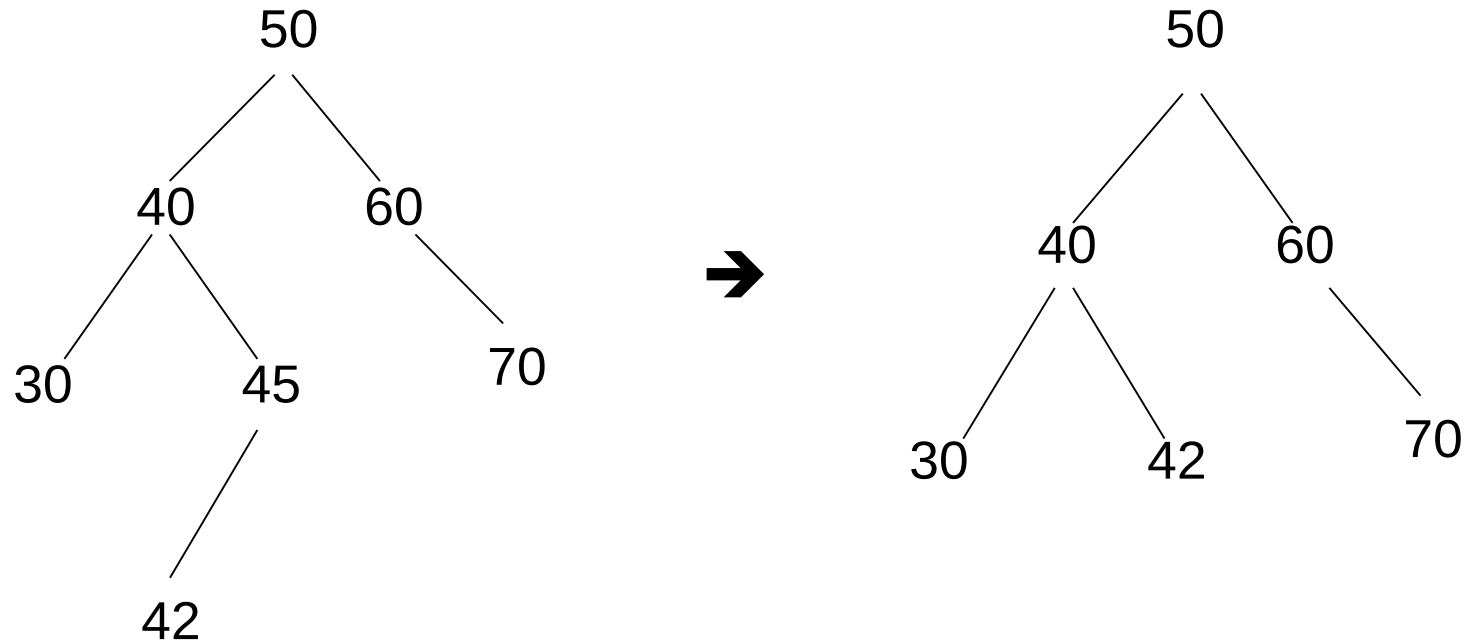
Deletion - Case1: A Leaf Node

To remove the leaf containing the item, we have to set the pointer in its parent to NULL.



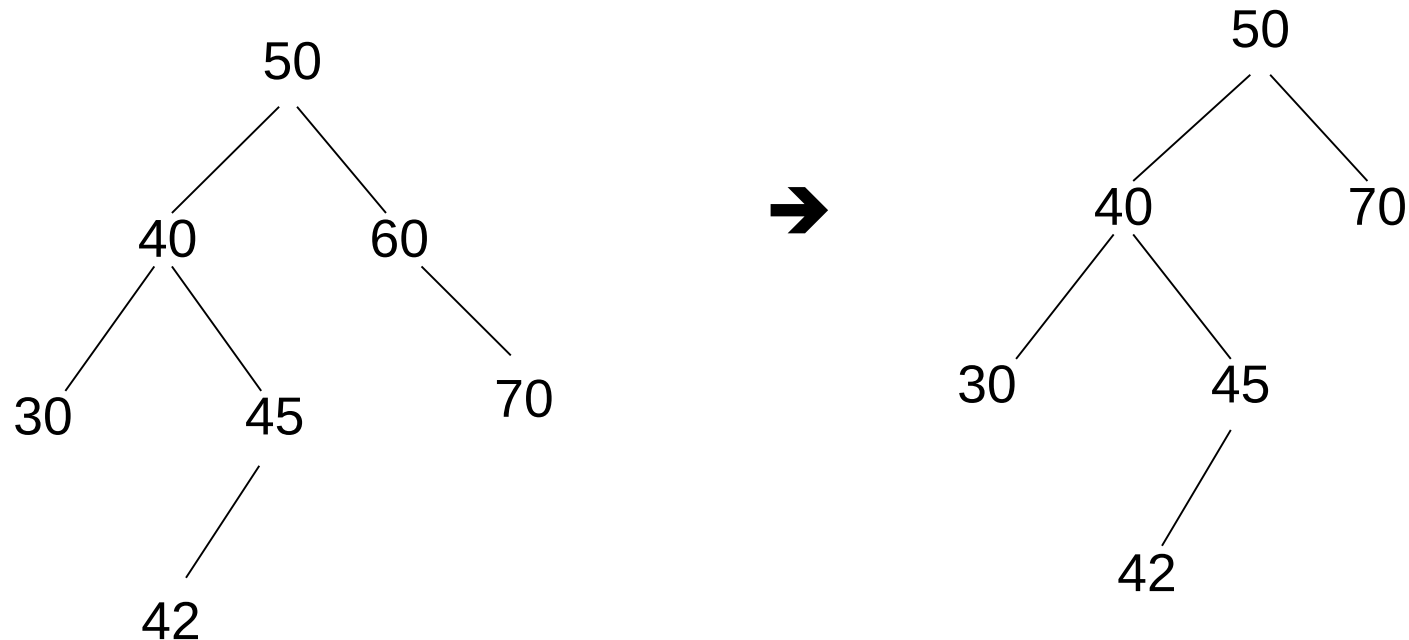
Deletion - Case2: A Node with only a left child

Delete 45 (A node with only a left child)



Deletion - Case2: A Node with only a right child

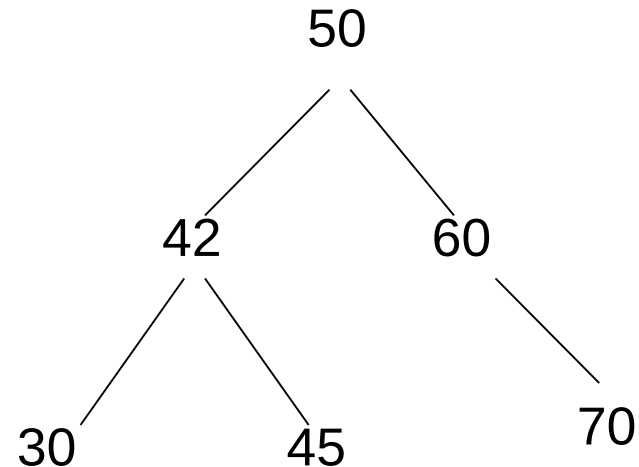
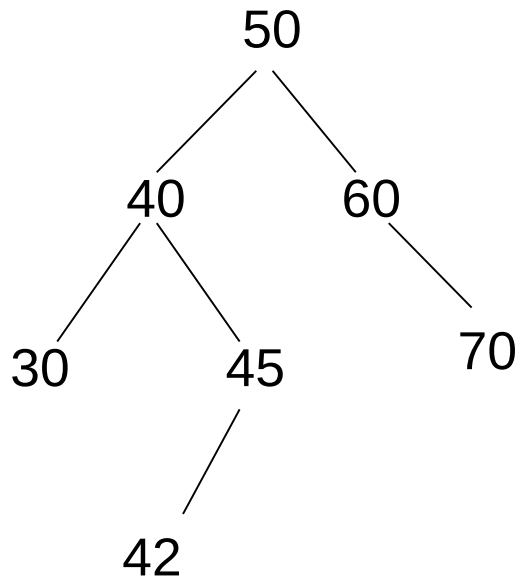
Delete 60 (A node with only a right child)



Deletion - Case3: A Node with two children

- Locate the inorder successor of the node.
- Copy the item in this node into the node which contains the item which will be deleted.
- Delete the node of the inorder successor.

Delete 40 (A node with two child)



```

SearchTree Delete( ElementType X, SearchTree T )
{
    Position TmpCell;

    if( T == NULL )
        Error( "Element not found" );
    else
        if( X < T->Element ) /* Go left */
            T->Left = Delete( X, T->Left );
        else
            if( X > T->Element ) /* Go right */
                T->Right = Delete( X, T->Right );
            else /* Found element to be deleted */
                if( T->Left && T->Right ) /* Two children */
                {
                    /* Replace with smallest in right subtree */
                    TmpCell = FindMin( T->Right );
                    T->Element = TmpCell->Element;
                    T->Right = Delete( T->Element, T->Right );
                }
            else /* One or zero children */
            {
                TmpCell = T;
                if( T->Left == NULL ) /* Also handles 0 children */
                    T = T->Right;
                else if( T->Right == NULL )
                    T = T->Left;
                free( TmpCell );
            }

    return T;
}

```

Operations on BST

- Most of the operations on binary trees are $O(\log N)$.
 - ❖ This is the main motivation for using binary trees rather than using ordinary lists to store items.
- Most of the operations can be implemented using recursion.
 - ❖ we generally do not need to worry about running out of stack space, since the average depth of binary search trees is $O(\log N)$.

Binary Search Tree - Best Time

- All BST operations are $O(d)$, where d is

$$d = \lfloor \log_2 N \rfloor$$

which is the depth of the BST with N nodes.

- ❖ What is the **best** case tree?
- ❖ What is the **worst** case tree?
- The best case running time of BST operations is $O(\log N)$

Binary Search Tree - Worst Time

- Worst case running time is $O(N)$
 - ❖ What happens when you Insert elements in ascending order?
 - Insert: 2, 4, 6, 8, 10, 12 into an empty BST
 - ❖ Problem: Lack of "balance"
 - Compare depths of left and right subtree
 - ❖ Unbalanced degenerate tree

Order of Operations on BSTs

<u>Operation</u>	<u>Average case</u>	<u>Worst case</u>
Retrieval	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$

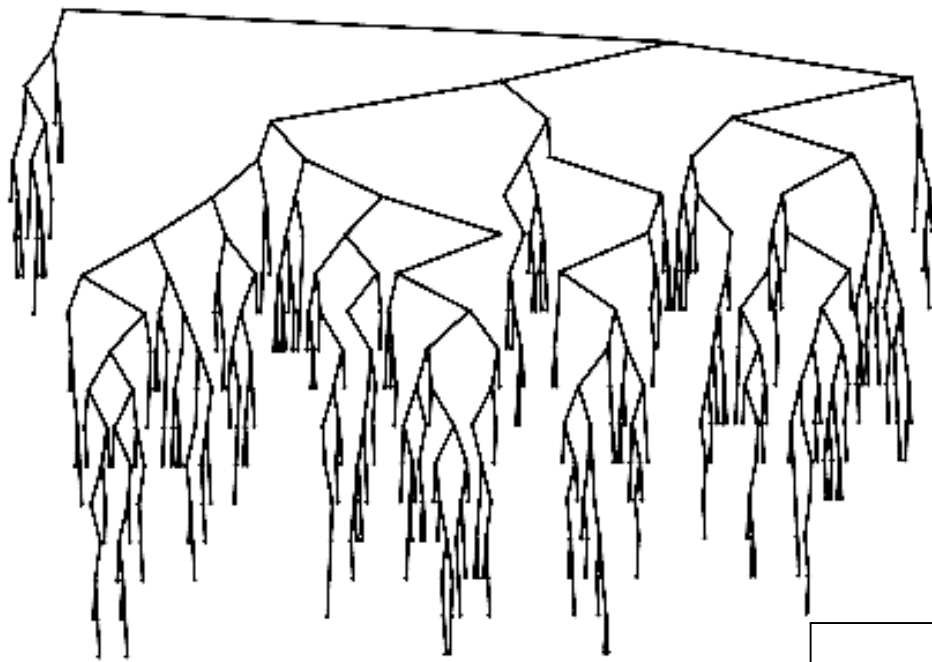
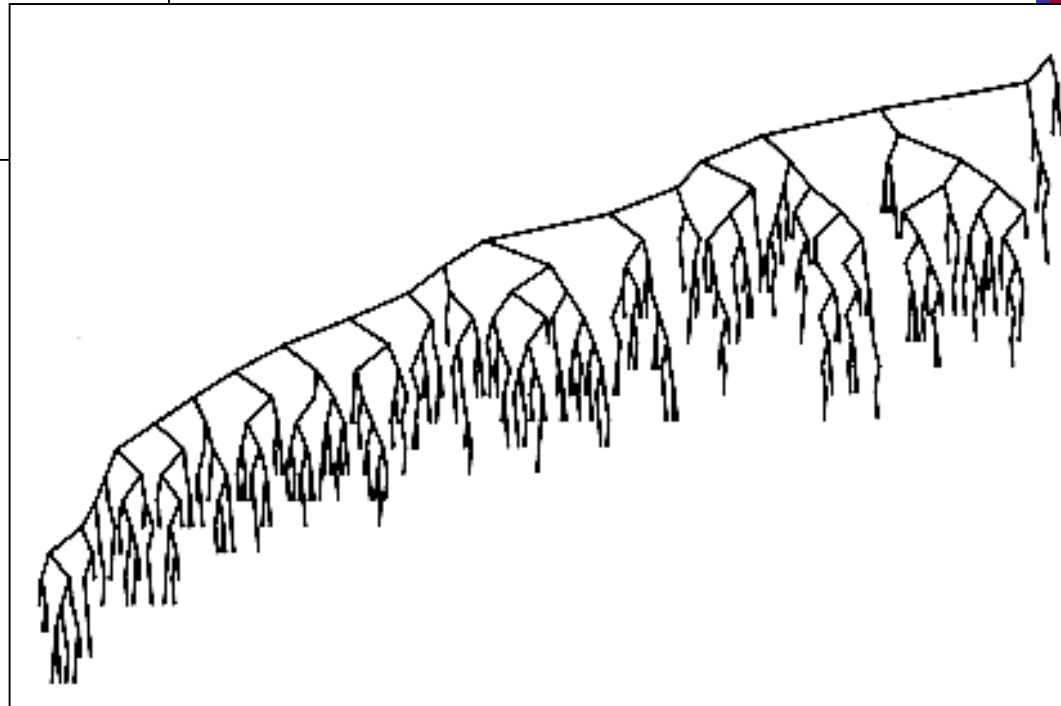


Figure 4.26 A randomly generated binary search tree



Algorithm Analysis And Data Structures

Questions and Answers

Thank you ...

Prof. Dr. Ayla ŞAYLI

[http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr](http://avesis.yildiz.edu.tr/sayli/sayli@yildiz.edu.tr)